Automated Black-Box Testing: A Comparative Study of LLM Agent Architectures and Prompt Engineering

Nguyen Van Thanh, Chen Enrico, Ma Xiaoning,
Ji XiaoQuan, Mai Minh Thai,

Correspondence: s336748@studenti.polito.it, s337750@studenti.polito.it, s337332@studenti.polito.it,
s339740@studenti.polito.it, s334001@studenti.polito.it

Abstract

Automated test case generation remains challenging due to diverse implementation behaviors and complex boundary conditions. This work investigates the effectiveness of Multi-agent *Large Language Model (LLM)* architectures employing collaborative and competitive interaction patterns, compared against a Single-agent baseline, within the context of black-box testing. Experiments on the HumanEval benchmark show that prompt engineering is the primary driver of performance, with *Augmented Few-Shot* prompting yielding $20 - 50\%$ both coverage and execution success rates improvements across all strategies and increasing code coverage from approximately 60% to nearly 99%. While a collaborative strategy optimized for execution success rate achieves the highest reliability (100% on curated subsets and 97.56% on the full dataset) it incurs a $3 - 4$ times increase in token consumption compared to Single-agent approaches. In contrast, a *Single-agent* configuration paired with optimized *Augmented Few-Shot* prompting provides the most effective balance between accuracy, coverage, and computational efficiency. These results indicate that Multi-agent collaboration is most beneficial for accuracy-critical scenarios, whereas well-engineered Single-agent workflows offer a more cost-effective solution for large-scale deployment in black-box testing contexts.

1 Introduction

Software testing often requires collaboration among developers, testers, domain experts, and automated tools, each contributing complementary perspectives. Developers focus on implementation choices, testers on edge cases and failure scenarios, and domain experts on realistic usage constraints. Effective testing depends on integrating these viewpoints to produce high-quality test suites.

Most automated test generation tools rely on a single reasoning strategy, limiting their ability to capture diverse behaviors, edge cases, and domain-specific knowledge. This limitation affects not only large systems but also individual functions, where subtle bugs can arise from boundary conditions or uncommon execution paths. Consequently, function-level testing remains essential, and its effectiveness strongly depends on the diversity and coverage of the generated test cases.

Recent advances in *LLMs* enable new approaches to automated test generation based on Multi-agent architectures. In such systems, multiple *LLM* agents can assume different roles, such as developer, tester, or domain expert, and collaborate to generate test cases for the same code under test. By combining complementary viewpoints, Multi-agent systems have the potential to produce more comprehensive and diverse test sets compared to Single-agent approaches.

In this work, we build upon the QA Agent framework proposed by (Deo, 2025), a Multi-agent system for unit test generation based on natural language pseudocode. We extend this framework by modifying the prompting strategies and agent interaction mechanisms to better suit our experimental setting. Our evaluation focuses on function-level test generation using the HumanEval benchmark under black-box testing conditions, comparing a Single-agent baseline with Multi-agent configurations that employ collaborative and competitive interaction patterns.

This study investigates the performance trade-offs between Multi-agent architectures and a Single-agent baseline. We aim to determine whether the improvements in execution success and code coverage offered by Multi-agent patterns justify their increased computational overhead. Furthermore, we quantify the extent to which advanced prompt engineering can bridge the performance gap between these configurations.

The quality of the generated test sets is evaluated using multiple criteria, including code coverage

metrics (line and branch coverage) and effectiveness measured by execution success rates against canonical solutions.

The full implementation of the proposed system is publicly available at https://github.com/Polito-MLDL-2025/30llm-testgen

1.1 Research Questions

In order to evaluate the capabilities of Multi-agent *LLM* architectures in generating high-quality test cases, we address the following research questions, which frame the objectives of this study:

- **RQ 1**: How effective are *LLM* agents in generating comprehensive and diverse test cases for a given set of functions?
- **RQ 2**: Does collaboration among multiple *LLM* agents improve test case generation compared to a Single-agent baseline?
- **RQ 3**: How do different patterns of collaboration impact the robustness of generated test cases, and what is the optimal strategy for test cases generation?

2 Background

With the introduction of the *attention mechanism* and the *transformer architecture* (Vaswani et al., 2023), the field of language models has seen noticeable improvements. In the *attention mechanism*, attention scores are computed and used to condition the output of the model on the relevant parts of the input (Vaswani et al., 2023). Furthermore, compared to previous architectures based on *RNN*, the *transformer architecture* has the advantage of being parallelizable; this means that multiple input tokens can be processed in a single step, and this increases efficiency (Minaee et al., 2025).

Large Language Models are transformer-based language models that have a number of parameters on the order of hundreds of billions (Minaee et al., 2025). Being pretrained on massive amounts of data, they show remarkable capabilities in understanding, reasoning, and generating natural language. They can be used in a huge variety of applications, ranging from *Natural Language Processing* tasks like *Text Summarization* and *Machine Translation* to *Software Engineering* tasks like code generation and test case generation. *LLMs* can be classified into three different types according to their architecture (Hou et al., 2024):

- *encoder-only*: adequate for tasks requiring understanding of the input like *code understanding*
- *encoder-decoder*: adequate for tasks requiring both understanding and generation capabilities like *code summarization*
- *decoder-only*: adequate for tasks requiring generation capabilities like *test case generation*

Being real-world problems inherently complex and requiring experts in multiple domains, performance of a *single LLM-based agent* can be limited, and in this context the introduction of *Multi-agent* systems marks an important evolution in the field, where by collaborating with each other and having distinct roles and responsibilities, agents can solve complex tasks (He et al., 2025) like Software Engineering tasks with improved factuality (Du et al., 2023). *Multi-agent* systems can collaborate with different patterns (Tran et al., 2025), the main ones are:

- *Cooperative*: agents cooperate with each other to reach a common goal
- *Competitive*: agents compete with each other to reach their own goal (for instance, with debate (Liang et al., 2024))

Throughout the years, substantial research has been conducted on *Multi-agent* systems applied to *Test case generation*. (Hong et al., 2024) simulates an entire software company where different agents with distinct roles cooperate with each other to create a software program. More specifically, there are 5 different roles: *Product manager* creates user stories and requirement pool given user requirements, *Architect* creates components, *Project manager* distributes the tasks, *Engineer* develops code, and *QA engineer* develops tests.

(Qian et al., 2024) is another *Multi-agent* system used for the design, coding, and testing phases of a software development. It divides each task into subtasks, and for each subtask there are 2 roles (*instructor*, *assistant*) communicating to each other to reach a consensus. It also introduces a mechanism where the *assistant* actively asks the *instructor* for clarifications to solve problems of factual correctness due to hallucinations (Zhang et al., 2025).

(Pan et al., 2025) uses the results of static analysis to guide *LLMs* in generating test cases. Another

work in which *LLMs* are guided is (Altmayer Pizzorno and Berger, 2025), in this case the prompt contains coverage information. (Wang et al., 2025) incorporates mutation feedback into the prompt, resulting in improved performance over standard prompt-based techniques.

(Huang et al., 2024) introduces a cooperative system with 3 agents: *programmer agent*, *test designer agent*, and *test executor agent*. The *programmer agent* is an *LLM* that creates functions according to human specifications using *Chain-Of-Thought* technique. The *test designer agent* is an *LLM* and generates test cases. Then, the *test executor agent*, which is not an *LLM*, tests the given code snippets with the test cases, if there are some failures, then the information is given to the *programmer agent* to refine the code. This system outperforms *Single-agent* systems and (Hong et al., 2024) in accuracy and line coverage, for both *HumanEval* (Chen et al., 2021) and *MBPP* (Austin et al., 2021).

(Deo, 2025) introduces a *Multi-agent* system where a *code architect agent* generates the function implementation plan in natural language and pseudocode, and a *test generator agent* generates the test cases. This system outperforms (Huang et al., 2024) in terms of coverage and accuracy for *HumanEval*, while for *MBPP* the accuracy is lower, the reason is that *HumanEval* contains at least one test case with a correct answer for each function.

(Chan et al., 2023) introduces a *competitive Multi-agent* system where agents debate in order to evaluate outputs, this debate mechanism can also be leveraged in *test case generation*.

3 System overview

This section details the architectures evaluated in our study, ranging from a baseline Single-agent configuration to complex Multi-agent collaborative and competitive frameworks. Our system is designed to automate unit test generation by leveraging the specialized capabilities of Large Language Models through structured role-playing and strategic task decomposition.

3.1 Single-Agent Baseline

The Single-agent approach serves as our control, utilizing a direct, one-step prompting strategy. In this configuration, a single model inference is tasked with generating a comprehensive test suite without intermediary planning.

Execution Flow: The Single-agent system follows a one-step generation pipeline:

> *Problem Description* $\rightarrow$ *Test Generator* $\rightarrow$ *Test Cases* $\rightarrow$ *Evaluation*

Prompted as a Quality Assurance Engineer, the model receives the function signature and docstring to produce executable Python unit tests. This approach performs no explicit strategic decomposition, allowing us to isolate the impact of agent collaboration in subsequent experiments. Tests are evaluated against canonical solutions to establish baseline metrics for line coverage and pass rates.

3.2 Multi-Agent System

This section details the *Multi-agent LLM* system designed for automated test case generation. The system leverages a specialized architecture where distinct agents work together to decompose the complex task of test generation into manageable sub-tasks: planning and execution. This separation of concerns aims to improve the quality, coverage, and accuracy of the generated tests compared to a Single-agent baseline.

The system operates as a linear pipeline where the output of one agent serves as the context-enhanced input for the next. This mimics a real-world software development workflow where a senior architect designs the solution (pseudocode/plan) and a developer (tester) implements the specific test cases based on that design.

Execution Flow:

> *Problem Description* $\rightarrow$ *Code Architect* $\rightarrow$ *Plan* $\rightarrow$ *Test Generator* $\rightarrow$ *Test Cases*$\rightarrow$ *Evaluation*

1. **Problem Description**: The system receives a coding problem definition (e.g., from HumanEval).
2. **Plan generation (Code Architect)**: The first agent analyzes the problem and produces a natural language plan or pseudocode.
3. **Test generation (Test Generator)**: The second agent receives the original problem plus the generated plan. It uses this context to write executable Python unit tests.
4. **Evaluation**: The generated tests are executed against the canonical solution to measure execution success rates (accuracy) and code coverage.

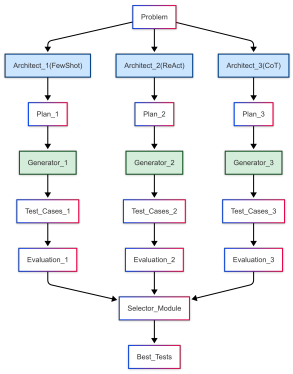

Figure 1: Multi-Agent Competitive Architecture

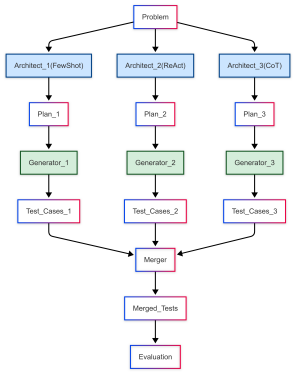

Figure 2: Multi-Agent Collaborative Architecture

This workflow is adapted from the original project. In the next sections, we introduce two new approaches that extend this pipeline to generate three test sets and apply different strategies to select the final test suite.

3.2.1 Multi-Agent Competitive Strategy

In the competitive Multi-agent paradigm, the system leverages diversity by executing independent generation pipelines in parallel. Rather than seeking a consensus or unified output, each Multi-agent operates autonomously to produce a distinct test suite, with the final output determined by a winner-take-all selection process. This workflow is illustrated in Figure 1.

Specifically, three independent pipelines are initialized, each utilizing a unique prompting style for the **Code Architect** (Chain-of-Thought, ReAct, and Few-Shot). Each resulting pseudocode plan informs the **Test Generator**, which produces a corresponding test suite. A final **Selector** module evaluates all three suites; the suite with the highest code coverage is chosen as the final output, with execution success rate serving as the tiebreaker. This strategy ensures a high-quality result by filtering for the most robust candidate while avoiding the logical conflicts or redundancies that can arise from merging disparate codebases.

3.2.2 Multi-Agent Collaborative Strategy

The collaborative strategy utilizes the same foundational Multi-agents and prompting configurations as the competitive model but adopts a different interaction paradigm centered on synthesis rather than selection. As illustrated in Figure 2, the workflow transitions from parallel exploration to a centralized reconciliation phase.

In this configuration, the three **Code Architect** agents generate diverse implementation plans. These plans are then processed by the **Test Generator** to produce individual test sets. A specialized **Merger** module is responsible for reconciling these outputs into a single cohesive unit.

Merging Mechanisms We evaluate two distinct strategies for reconciling the multi-agent outputs:

- **Concatenation-Based**: This strategy aggregates all generated test cases into a single suite. It prioritizes the volume and diversity of the executable set.
- **Accuracy-Based**: This approach analyzes and selects the most effective tests. It prioritizes execution success rate and logical consistency, pruning tests like those with syntax errors or identical duplicates detected via AST comparison or wrong function names.

3.3 Prompt Engineering

In addition to exploring different architectures for the pipeline, we also investigate how prompt engineering influences the overall system performance.

For each agent in the system, we evaluate multiple role-specific prompting strategies.

For the *Code Architect*, responsible for high-level implementation planning, we consider three prompting styles: a basic Chain-of-Thought (CoT) without examples, an anchored CoT with few-shot examples, and a ReAct-style workflow that enforces an explicit "Thought-Act" structure with few-shot examples. While the standard QA Agent relies on Prompt CoT featuring few-shot examples, our Competitive and Merge variants utilize all three styles to explore diverse planning strategies.

For the *Test Generator*, which facilitate the production of test cases, we examine three prompt configurations. The *Zero-Shot* configuration provides only the task definition without any contex-

tual examples, relying entirely on the model's innate reasoning ability to generate test cases based on the function signature and description. This serves as a baseline to evaluate the model's raw performance. The *Standard Few-Shot* configuration, adopted by the original QA Agent framework, employs a role-based structure with few-shot examples drawn from related tasks, offering additional contextual guidance. The *Augmented Few-Shot* configuration further extends this approach by incorporating detailed guidelines, including rules for numeric precision, boundary logic, and deterministic outputs, along with few-shot examples. This configuration helps the model develop a more comprehensive understanding of the task, guiding it to generate high-quality test cases.

4 Experimental results

4.1 Experimental Setup

This section describes the experimental framework used to evaluate the efficacy of the Multi-agent system in generating unit tests.

4.1.1 Datasets and Tasks

The system is evaluated on the HumanEval benchmark, which consists of 164 Python programming tasks. Each problem includes a function signature, a descriptive docstring, a reference (canonical) implementation, and unit tests.

To ensure both depth and breadth in our evaluation, we employ two experimental configurations. First, we use a curated subset of 20 HumanEval problems, spanning from basic arithmetic to advanced logic validation. These tasks are categorized as follows: Basic (5 tasks), Medium (8 tasks), Hard (3 tasks), and Advanced (4 tasks). Each task is executed 10 times per strategy, ensuring statistical robustness and minimizing the impact of variance in model outputs. Second, we assess the system on the full set of 164 HumanEval problems, executed once per strategy, to evaluate the generalizability of our approach across a wide range of programming tasks.

For each task, we extract three core components to facilitate the generation and testing workflow. The *Prompt* (signature and docstring) defines the model's input, the *Canonical Solution* provides a reference for correctness, and the *Entry Point* specifies the function name used for automated unit testing.

4.1.2 Evaluation Metrics

We assess the quality and efficiency of the generated test suites using three primary metrics: *Code Coverage*, *Effectiveness*, and *Token Consumption*. *Code Coverage* quantifies the degree to which the generated tests execute the logical paths and conditions within the canonical solution, serving as a measure of test thoroughness. *Effectiveness* evaluates the validity of the tests by verifying their success rate when executed against the ground-truth reference implementation. Finally, we monitor *Token Consumption* by calculating the average total tokens (input and output) processed per task, providing a baseline for the computational efficiency and cost of each strategy.

4.1.3 Model Selection

We utilize the *nvidia/nemotron-3-nano-30b-a3b* model, a Mixture-of-Experts (MoE) architecture with $3.5B$ active and $30B$ total parameters. Selected for its efficiency and specialized fine-tuning in code and structured outputs, the model is run at a fixed temperature of 0. By maintaining a single, consistent model across all configurations, we isolate the effects of our prompting and agent strategies, ensuring that performance variations are attributable solely to our methodological variants. This approach ensures a fair comparative analysis.

4.1.4 Strategies and Variants

We evaluate four architectural strategies for test generation. The *Single-agent* approach serves as the baseline, generating test cases directly using a single *LLM*. The *QA Agent* extends this baseline with a Multi-agent architecture derived from the QA Agent framework. The *QA Agent Competitive* variant introduces a competitive interaction pattern in which multiple candidate test suites are generated, and the best-performing one is selected. The *QA Agent Merge* strategy aggregates outputs from multiple agents using either an *Accuracy*-based or *Concat*-based merging scheme.

All strategies are evaluated using both *Standard Few-Shot* and *Augmented Few-Shot* prompting, while the Single-agent configuration additionally includes a *Zero-Shot* setup as a baseline.

4.2 Results and Analysis

The results of our evaluation on the HumanEval benchmark are summarized in Table 1 and Table 2. These data points illustrate the performance tradeoffs between prompting styles and Multi-agent

strategies.

The Primacy of Prompt Engineering: Across all architectural strategies, the *Augmented Few-Shot* (optimized) prompt consistently outperforms the *Standard Few-Shot* and *Zero-Shot* configurations. Notably, the shift from *Zero-Shot* to *Augmented Few-Shot* yields improvements of $20 - 50\%$ in both coverage and execution success rates, demonstrating that prompt refinement is as critical to performance as the underlying agent strategy.

Strategy Performance vs. Complexity: The *QA Agent Merge (Accuracy)* strategy establishes the performance ceiling, achieving a perfect execution success rate (100%) on the curated subset and 97.56% on the full dataset. However, this reliability incurs a substantial overhead: Multi-agent architectures require approximately $3 - 4$ times the token volume of the Single Agent baseline. This identifies a clear trade-off where marginal gains in accuracy come at a non-linear computational cost.

Efficiency and Generalizability: The *Single Agent (Augmented Few-Shot)* configuration proves to be the most cost-effective for general-purpose applications, achieving an impressive execution success rate of 97.24% and the highest code coverage on the full dataset at 98.77%, all while minimizing token usage. However, for tasks where accuracy is paramount, the *QA Agent Merge (Augmented Few-Shot - Accuracy)* variant is the preferred option, offering better precision.

Our analysis indicates a clear hierarchy of needs in automated test generation. While Multi-agent collaboration (specifically the *Merge* strategy) is superior for mission-critical tasks where maximum accuracy is required, the *Single Agent* approach with optimized prompting offers the best balance of performance and resource efficiency for large-scale deployments.

5 Conclusion

In this work, we explore the effectiveness of *Multi-agent LLM* architectures in automated black-box test generation.

Based on our experiments, we provide the following answers to our research questions:

RQ 1: Effectiveness of LLM Agents. Our findings indicate that *LLM* agents are highly effective at generating diverse test cases, provided that prompt engineering is optimized. Prompt design emerges as a critical factor influencing generation quality. The *Augmented Few-Shot* prompting strategy consistently outperforms *Zero-Shot* and *Standard Few-Shot* configurations. Since the model, *nvidia/nemotron-3-nano-30b-a3b*, is specifically optimized for coding tasks, the *Augmented Few-Shot* prompt enables a single agent to generate high-quality test cases efficiently without requiring additional steps.

RQ 2: Impact of Collaboration. Collaboration demonstrates clear superiority over the standard baseline. Multi-agent strategies outperform both the *Zero-Shot* and *Standard Few-Shot* Single-agent configurations in terms of Coverage and Execution Success Rate. This indicates that collaborative paradigms leverage diverse reasoning to achieve higher robustness and comprehensiveness, particularly where basic Single-agent setups fail to handle complexity.

RQ 3: Optimal Strategies and Robustness. In terms of collaboration patterns, the *QA Agent Merge (Accuracy)* approach achieves the highest execution success rate, making it the most robust strategy for complex test generation, albeit at a higher computational cost. However, due to the black-box nature of our testing, complex feedback-driven revisions prove ineffective. Consequently, the *Single Agent* approach combined with *Augmented Few-Shot* prompting provides the optimal balance between performance and resource efficiency for large-scale, general-purpose applications.

6 Future Work

Several extensions can enhance the effectiveness and scalability of Multi-agent test generation systems. Expanding the evaluation to datasets like *MBPP* will allow for a broader assessment of the approach's generalizability across diverse tasks. Additionally, enhancing competitive strategies by incorporating additional models and multiple refinement loops could further optimize the generated test cases. The system could also evolve into a more advanced Multi-agent collaborative framework, consisting of components such as an agent for codebase understanding, a tester planner to generate and refine test plans, a test generator to create test cases, and an evaluation process that identifies weaknesses and triggers necessary refinements. By integrating these stages, the system will continuously improve, enabling the autonomous generation of high-quality, comprehensive test suites.

References

Juan Altmayer Pizzorno and Emery D. Berger. 2025. Coverup: Effective high coverage test generation for python. *Proceedings of the ACM on Software Engineering*, 2(FSE):2897–2919.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models. *Preprint*, arXiv:2108.07732.

Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. 2023. Chateval: Towards better llm-based evaluators through multi-agent debate. *Preprint*, arXiv:2308.07201.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code. *Preprint*, arXiv:2107.03374.

Akhil Deo. 2025. Qaagent: a multiagent system for unit test generation via natural language pseudocode (student abstract). In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'25/IAAI'25/EAAI'25. AAAI Press.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. *Preprint*, arXiv:2305.14325.

Junda He, Christoph Treude, and David Lo. 2025. Llm-based multi-agent systems for software engineering: Literature review, vision and the road ahead. *Preprint*, arXiv:2404.04834.

Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. Metagpt: Meta programming for a multi-agent collaborative framework. *Preprint*, arXiv:2308.00352.

Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *Preprint*, arXiv:2308.10620.

Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2024. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *Preprint*, arXiv:2312.13010.

Tian Liang, Zhiwei He, Wenxiang Jiao, Xing Wang, Yan Wang, Rui Wang, Yujiu Yang, Shuming Shi, and Zhaopeng Tu. 2024. Encouraging divergent thinking in large language models through multi-agent debate. *Preprint*, arXiv:2305.19118.

Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2025. Large language models: A survey. *Preprint*, arXiv:2402.06196.

Rangeet Pan, Myeongsoo Kim, Rahul Krishna, Raju Pavuluri, and Saurabh Sinha. 2025. Aster: Natural and multi-language unit test generation with llms. *Preprint*, arXiv:2409.03093.

Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. Chatdev: Communicative agents for software development. *Preprint*, arXiv:2307.07924.

Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, and Hoang D. Nguyen. 2025. Multi-agent collaboration mechanisms: A survey of llms. *Preprint*, arXiv:2501.06322.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention is all you need. *Preprint*, arXiv:1706.03762.

Guancheng Wang, Qinghua Xu, Lionel C. Briand, and Kui Liu. 2025. Mutation-guided unit test generation with a large language model. *Preprint*, arXiv:2506.02954.

Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Chen Xu, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. 2025. Siren's song in the ai ocean: A survey on hallucination in large language models. *Preprint*, arXiv:2309.01219.

A Appendix

Strategy	Prompt	Execution Success Rate (%)	Coverage (%)	Tokens
Single Agent	Zero Shot (Baseline)	48.66	57.15	47,511.50
Single Agent	Standard Few-Shot	78.89	84.66	49,949.10
QA Agent	Standard Few-Shot	71.76	93.86	73,580.40
QA Agent Competitive	Standard Few-Shot	87.17	90.08	245,753.70
QA Agent Merge (Accuracy)	Standard Few-Shot	91.00	88.87	235,632.30
QA Agent Merge (Concat)	Standard Few-Shot	72.94	93.58	241,940.20
Single Agent	Augmented Few-Shot	98.48	98.92	82,873.70
QA Agent	Augmented Few-Shot	97.65	**99.40**	106,228.40
QA Agent Competitive	Augmented Few-Shot	98.93	99.20	333,891.50
QA Agent Merge (Accuracy)	Augmented Few-Shot	**100.00**	99.39	333,524.30
QA Agent Merge (Concat)	Augmented Few-Shot	97.61	99.25	334,223.40

Table 1: Evaluation results on the HumanEval curated subset (20 problems), sorted by Prompt and Strategy. Results represent the average performance across 10 independent runs per strategy to ensure statistical robustness.

Strategy	Prompt	Execution Success Rate (%)	Coverage (%)	Tokens
Single Agent	Zero Shot (Baseline)	60.61	68.11	362,462.00
Single Agent	Standard Few-Shot	86.65	91.93	405,078.00
QA Agent	Standard Few-Shot	82.77	95.43	586,740.00
QA Agent Competitive	Standard Few-Shot	93.17	97.31	1,822,078.00
QA Agent Merge (Accuracy)	Standard Few-Shot	96.95	96.62	1,890,739.00
QA Agent Merge (Concat)	Standard Few-Shot	81.17	93.82	1,845,532.00
Single Agent	Augmented Few-Shot	97.24	**98.77**	641,483.00
QA Agent	Augmented Few-Shot	96.50	98.57	824,319.00
QA Agent Competitive	Augmented Few-Shot	96.92	97.92	2,564,180.00
QA Agent Merge (Accuracy)	Augmented Few-Shot	**97.56**	97.35	2,560,891.00
QA Agent Merge (Concat)	Augmented Few-Shot	95.41	98.27	2,575,158.00

Table 2: Evaluation results on the full HumanEval dataset (164 problems), sorted by Prompt and Strategy. Results represent a single execution per strategy to assess broad generalizability across all benchmark tasks.